

Instrukcja uruchomienia chatbota — Windows 10/11

Chatbot pozwala zadawać pytania w języku polskim o dane produkcyjne zapisane w plikach Excel. Sztuczna inteligencja przeszukuje dokumenty i odpowiada na podstawie rzeczywistych danych — wszystko działa lokalnie, bez internetu i bez wysyłania danych na zewnątrz.

Po wykonaniu tej instrukcji zobaczysz interfejs chatbota w przeglądarce i będziesz mógł zadawać pytania o wady produkcyjne drzwi.

Spis treści

- 1. Wymagania sprzętowe
- 2. Instalacja Docker Desktop
- 3. Instalacja Ollama
- 4. Pobranie projektu
- 5. Konfiguracja pliku .env
- 6. Dodanie pliku Excel z danymi
- 7. Uruchomienie aplikacji
- 8. Korzystanie z chatbota
- 9. Zatrzymanie i ponowne uruchomienie
- 10. Rozwiązywanie problemów

1. Wymagania sprzętowe

Przed rozpoczęciem sprawdź, czy Twój komputer spełnia poniższe wymagania:

Wymaganie	Minimum	Zalecane
System operacyjny	Windows 10 (64-bit, wersja 2004+)	Windows 11
RAM	8 GB	16 GB
Wolne miejsce na dysku	20 GB	30 GB
Procesor	x86-64, 4 rdzenie	x86-64, 8 rdzeni
Karta graficzna	brak wymagań	NVIDIA (przyspiesza AI)

Czas pierwszego uruchomienia: około 15–30 minut (jednorazowe pobieranie modeli AI, rozmiar ~2–7 GB w zależności od wybranego modelu).

Wskazówka dotycząca RAM:

Jeśli Twój komputer ma 8 GB RAM — wybierz model `llama3.2:3b` (2 GB, szybki, angielski).
Jeśli masz 16 GB RAM — możesz użyć `SpeakLeash/bielik-11b-v2.3-instruct:Q4_K_M` (7 GB, pełny polski model).
Szczegóły w sekcji [Konfiguracja pliku .env](#).

2. Instalacja Docker Desktop

Docker to program, który uruchamia aplikację w izolowanych "kontenerach" — dzięki temu nie musisz ręcznie instalować dziesiątek bibliotek i baz danych.

Krok 1 — Pobierz Docker Desktop

Wejdź na stronę: <https://www.docker.com/products/docker-desktop>

Kliknij **"Download for Windows"**. Zostanie pobrany plik instalacyjny (ok. 500 MB).

Krok 2 — Zainstaluj Docker Desktop

1. Uruchom pobrany plik `Docker Desktop Installer.exe`
2. Jeśli pojawi się okno "Kontrola konta użytkownika" — kliknij **Tak**
3. W oknie instalatora upewnij się, że zaznaczona jest opcja **"Use WSL 2 instead of Hyper-V"** (WSL 2 to podsystem Linux dla Windows — Docker go potrzebuje)
4. Kliknij **Ok** i poczekaj na zakończenie instalacji (~2–5 minut)
5. Kliknij **Close and restart** — komputer się uruchomi ponownie

Krok 3 — Uruchom Docker Desktop

Po restarcie:

1. Znajdź **Docker Desktop** w menu Start i uruchom go
2. W zasobniku systemowym (prawa strona paska zadań, przy zegarze) pojawi się ikona wieloryba
3. Poczekaj aż ikona przestanie się animować i pojawi się komunikat **"Docker Desktop is running"**

Jeśli przy pierwszym uruchomieniu pojawi się okno z prośbą o akceptację warunków — kliknij **Accept**.

Krok 4 — Zweryfikuj instalację

Otwórz **PowerShell** (wyszukaj "PowerShell" w menu Start) i wpisz:

```
docker --version
```

Powinieneś zobaczyć coś w stylu: `Docker version 27.x.x, build ...`

Jeśli zamiast tego widzisz błąd — upewnij się, że Docker Desktop jest uruchomiony (ikona wieloryba w zasobniku).

3. Instalacja Ollama

Ollama to program, który pobiera i uruchamia modele językowe AI lokalnie na Twoim komputerze. Chatbot łączy się z Ollamą, żeby generować odpowiedzi.

Krok 1 — Pobierz Ollama

Wejdź na stronę: <https://ollama.com>

Kliknij **"Download"** → wybierz **Windows**. Zostanie pobrany plik `OllamaSetup.exe` (ok. 100 MB).

Krok 2 — Zainstaluj Ollama

1. Uruchom `OllamaSetup.exe`
2. Jeśli pojawi się okno "Kontrola konta użytkownika" — kliknij **Tak**
3. Kliknij **Next** → **Next** → **Install** → **Finish**

Ollama zainstaluje się i automatycznie uruchomi w tle jako usługa systemowa.

Krok 3 — Zweryfikuj instalację

Otwórz **PowerShell** i wpisz:

```
ollama --version
```

Powinieneś zobaczyć numer wersji, np.: `ollama version 0.x.x`

Krok 4 — Uruchom serwer Ollama

Otwórz **nowe okno PowerShell** i wpisz:

```
ollama serve
```

Zostawcie to okno otwarte — serwer Ollama musi działać przez cały czas korzystania z chatbota.

Ważne: Jeśli Ollama jest już uruchomiona jako usługa w tle, polecenie `ollama serve` może wyświetlić błąd "address already in use" — to jest normalne i oznacza, że Ollama już działa. Możesz to okno zamknąć.

Sprawdź czy Ollama odpowiada — wpisz w PowerShell:

```
curl http://localhost:11434/api/tags
```

Jeśli zobaczysz odpowiedź JSON (nawet pustą `{"models":[]}`), Ollama działa poprawnie.

4. Pobranie projektu

Opcja A — Przez Git (zalecana)

Jeśli masz zainstalowany Git (<https://git-scm.com/download/win>):

```
git clone <URL-repozytorium>
cd door-defects-chatbot
```

(Zastąp `<URL-repozytorium>` adresem podanym przez prowadzącego)

Opcja B — Przez ZIP z GitHub

1. Wejdź na stronę repozytorium na GitHub
2. Kliknij zielony przycisk **Code** → **Download ZIP**
3. Wyodrębnij pobrany archiwum w wybranym miejscu (np. `C:\Users\TwojeImię\door-defects-chatbot`)

Przejdź do folderu projektu

Otwórz **PowerShell** i przejdź do folderu projektu:

```
cd C:\ścieżka\do\door-defects-chatbot
```

Przykład: jeśli wyodrębniłeś ZIP do folderu Pobrane:

```
cd "$env:USERPROFILE\Downloads\door-defects-chatbot"
```

Sprawdź, że jesteś we właściwym miejscu — powinny być widoczne pliki projektu:

```
dir
```

Powinieneś zobaczyć m.in.: `docker-compose.yml`, `.env.example`, `backend\`, `frontend\`, `data\`

5. Konfiguracja pliku .env

Plik `.env` zawiera ustawienia aplikacji — m.in. jaki model AI zostanie użyty.

Krok 1 — Utwórz plik .env

```
copy .env.example .env
```

Krok 2 — Wybierz model AI (opcjonalnie)

Otwórz plik `.env` w Notatniku:

```
notepad .env
```

Znajdź linię zaczynającą się od `LLM_MODEL=` i wybierz jeden z modeli:

Model	Język odpowiedzi	Rozmiar	Min. RAM	Kiedy wybrać
<code>SpeakLeash/bielik-11b-v2.3-instruct:Q4_K_M</code>	Polski	~7 GB	16 GB	Masz 16+ GB RAM, chcesz odpowiedzi po polsku
<code>llama3.1:8b</code>	Angielski	~5 GB	8 GB	Masz 8–16 GB RAM
<code>llama3.2:3b</code>	Angielski	~2 GB	8 GB	Masz 8 GB RAM, chcesz szybkich odpowiedzi

Przykład dla 8 GB RAM:

```
LLM_MODEL=llama3.2:3b
```

Nie zmieniaj linii `OLLAMA_BASE_URL=http://host.docker.internal:11434` — ten adres jest poprawny dla Dockera na Windows.

Zapisz plik (`Ctrl+S`) i zamknij Notatnik.

6. Dodanie pliku Excel z danymi

Chatbot analizuje dane z plików Excel. Pliki muszą znajdować się w folderze `data\excel\`.

Struktura folderu

```
door-defects-chatbot\  
└─ data\  
    └─ excel\  
        └─ raport wad marzec 2026 - katalog_PowerBI.xlsx ← przykładowy plik
```

Przykładowy plik jest już w projekcie — możesz od razu uruchomić aplikację bez dodawania niczego.

Dodawanie własnych plików

Skopiuj swoje pliki `.xlsx` do folderu `data\excel\`. Każdy wiersz arkusza staje się osobnym dokumentem w bazie wiedzy chatbota.

Po dodaniu pliku podczas działania aplikacji — kliknij **menu (≡)** → **Przeglądaj dane Excel**, żeby chatbot zaindeksował nowe dokumenty.

Ponowne wrzucenie tego samego pliku **nie zduplikuje danych** — system sprawdza, które dokumenty już są w bazie.

7. Uruchomienie aplikacji

Krok 1 — Upewnij się, że Docker i Ollama działają

- Ikona wieloryba Docker w zasobniku systemowym = Docker działa
- W PowerShell: `curl http://localhost:11434/api/tags` zwraca odpowiedź = Ollama działa

Krok 2 — Uruchom aplikację

W PowerShell, w folderze projektu:

```
docker compose up --build
```

Jeśli PowerShell wyświetli błąd uprawnień, uruchom go jako Administrator:
kliknij prawym przyciskiem na ikonę PowerShell w menu Start → **Uruchom jako administrator**

Co się dzieje podczas uruchamiania

Zobaczysz dużo tekstu w oknie PowerShell — to normalne. Aplikacja uruchamia się etapami:

Etap 1 — Budowanie obrazów Docker (~2–5 min przy pierwszym uruchomieniu):

```
[backend] Building...
[frontend] Building...
```

Etap 2 — Uruchamianie kontenerów:

```
Container door-chromadb Started ← baza wektorowa gotowa
Container door-backend Starting ← serwer Python się ładuje
```

Etap 3 — Pobieranie modeli AI (tylko pierwsze uruchomienie, ~10–20 min):

```
Pobieranie modelu: llama3.2:3b (to może potrwać kilka minut)...
```

Etap 4 — Aplikacja gotowa:

```
door-backend | INFO: Application startup complete.
door-frontend | Starting...
```

Krok 3 — Otwórz chatbota

Gdy w logach pojawi się `Application startup complete.`, otwórz przeglądarkę i wejdź na:

`http://localhost:3000`

Powinieneś zobaczyć interfejs chatbota z polem do wpisywania pytań.

8. Korzystanie z chatbota

Wskaźnik statusu

W prawym górnym rogu ekranu znajduje się wskaźnik statusu:

- **Zielony** — wszystko działa, chatbot gotowy do odpowiedzi
- **Żółty** — model AI się ładuje, poczekaj chwilę
- **Czerwony** — problem z połączeniem (sprawdź Ollamę — patrz [sekcja 10](#))

Zadawanie pytań

Wpisz pytanie w języku polskim (lub angielskim, jeśli wybrałeś model llama) w polu na dole ekranu i naciśnij `Enter`.

Przykładowe pytania:

- "Jakie są najczęstsze wady powierzchni?"
- "Który wydział ma najwięcej reklamacji?"
- "Ile sztuk wadliwych odnotowano łącznie?"
- "Jakie wady dotyczą futryn?"

Chatbot wyświetla odpowiedź słowo po słowie (jak pisanie w czasie rzeczywistym) i na końcu pokazuje fragmenty dokumentów, z których skorzystał.

Skróty klawiszowe

Skrót	Akcja
Enter	Wyślij wiadomość
Shift+Enter	Nowa linia (bez wysyłania)
Ctrl+K	Nowy czat (wyczyść historię)
Esc	Zamknij menu boczne

Menu boczne (≡)

Kliknij ikonę menu w lewym górnym rogu, żeby:

- **Przeglądaj dane Excel** — zaindeksuj nowe pliki wrzucone do `data\excel\`
- **Nowy czat** — wyczyść historię rozmowy

9. Zatrzymanie i ponowne uruchomienie

Zatrzymanie aplikacji

W oknie PowerShell gdzie działa aplikacja naciśnij `Ctrl+C`, lub wpisz w nowym oknie:

```
docker compose down
```

Dane (zaindeksowane dokumenty) są zachowane na dysku.

Ponowne uruchomienie

Przy kolejnym uruchomieniu modele AI już są pobrane — start zajmie tylko ~30–60 sekund:

```
docker compose up
```

(bez `--build` — obrazy Docker też są zbudowane, nie trzeba od nowa)

Całkowity reset (usuwa bazę wiedzy)

Jeśli chcesz zacząć od zera — usunąć wszystkie zaindeksowane dokumenty:

```
docker compose down -v
docker compose up --build
```

Uwaga: `-v` usuwa woluminy Docker, w tym bazę wektorową ChromaDB. Modele Ollama NIE są usuwane — te są przechowywane przez Ollamę poza Dockerem.

10. Rozwiązywanie problemów

Problem: Chatbot pokazuje czerwony wskaźnik statusu

Objawy: Status "Offline" lub "Błąd połączenia", chatbot nie odpowiada.

Przyczyna: Ollama nie działa lub jest niedostępna.

Rozwiązanie:

1. Otwórz PowerShell i wpisz:

```
powershell ollama serve
```

2. Jeśli widzisz błąd "address already in use" — Ollama już działa, problem leży gdzie indziej.

3. Sprawdź czy Ollama odpowiada:

```
powershell curl http://localhost:11434/api/tags
```

4. Jeśli nie odpowiada — zrestartuj Ollamę: wyszukaj "Usługi" w menu Start, znajdź "Ollama", kliknij prawym → Uruchom ponownie.

Problem: Docker Desktop nie startuje (błąd WSL2)

Objawy: Komunikat "WSL 2 installation is incomplete" lub podobny.

Rozwiązanie:

1. Otwórz PowerShell **jako Administrator** (prawy przycisk myszy → Uruchom jako administrator)

2. Wpisz:

```
powershell wsl --install
```

3. Uruchom ponownie komputer

4. Po restarcie uruchom Docker Desktop

Problem: Port jest zajęty ("address already in use")

Objawy: Błąd podczas `docker compose up`, np.:

```
Error: bind: address already in use (port 3000)
```

Rozwiązanie:

Znajdź i zamknij program używający portu:

```
# Znajdź PID procesu używającego portu (np. 3000)
netstat -ano | findstr :3000

# Zamknij ten proces (zastąp 12345 numerem PID z poprzedniego polecenia)
taskkill /PID 12345 /F
```

Dla portu 8080 (backend) lub 8001 (ChromaDB) — powtórz te same kroki ze zmienionym numerem portu.

Problem: Backend ciągle się restartuje (brak pamięci)

Objawy: W logach pojawia się `Restarting...` co kilka sekund, backend nie działa stabilnie.

Przyczyna: Za mało RAM dla wybranego modelu AI.

Rozwiązanie:

1. Otwórz plik `.env` w Notatniku:

```
powershell notepad .env
```


2. Zmień linię `LLM_MODEL=` na mniejszy model:

```
LLM_MODEL=llama3.2:3b
```

3. Zapisz plik i uruchom ponownie:

```
powershell docker compose down docker compose up
```

Problem: Brak pliku `.env`

Objawy: Backend nie startuje, w logach błąd o brakujących zmiennych środowiskowych.

Rozwiązanie:

```
copy .env.example .env
```

Problem: Frontend pokazuje błąd 502 (Bad Gateway)

Objawy: Przeglądarka pokazuje "502 Bad Gateway" lub "Error connecting to backend".

Przyczyna: Backend jeszcze się ładuje (trwa indeksowanie plików lub ładowanie modelu).

Rozwiązanie: Oczekaj 1–3 minuty i odśwież stronę (`F5`). Jeśli błąd nie znika:

```
docker compose logs backend
```

Sprawdź ostatnie linie — szukaj błędów lub informacji o problemie.

Problem: Chatbot odpowiada "Nie znalazłem tej informacji"

Przyczyna: Brak zaindeksowanych danych lub pytanie dotyczy tematu poza plikami Excel.

Rozwiązanie:

1. Sprawdź czy pliki `.xlsx` są w folderze `data\excel\`
 2. W aplikacji kliknij `≡` → **Przeładuj dane Excel**
 3. Sprawdź statystyki bazy: otwórz w przeglądarce `http://localhost:8080/stats` — powinna pokazać liczbę dokumentów `> 0`
-

Problem: Bardzo wolne odpowiedzi (ponad 60 sekund)

Przyczyna: Model AI działa na procesorze (CPU) zamiast karcie graficznej — to normalne bez GPU NVIDIA.

Rozwiązanie A: Użyj mniejszego modelu — edytuj `.env` :

```
LLM_MODEL=llama3.2:3b
```

Rozwiązanie B: Jeśli masz kartę NVIDIA — zainstaluj sterowniki CUDA i upewnij się, że Docker Desktop ma włączoną obsługę GPU.

Sprawdzenie logów

Gdy coś nie działa, logi są najlepszym źródłem informacji:

```
# Logi wszystkich kontenerów
docker compose logs

# Logi konkretnego kontenera
docker compose logs backend
docker compose logs frontend
docker compose logs chromadb

# Logi na żywo (aktualizują się w czasie rzeczywistym)
docker compose logs -f backend
```

Naciśnij `Ctrl+C`, żeby zatrzymać podgląd logów na żywo.

Porty aplikacji

Serwis	Adres w przeglądarce	Do czego
Chatbot (interfejs)	<code>http://localhost:3000</code>	Używasz na co dzień
Backend API	<code>http://localhost:8080</code>	Dokumentacja API
ChromaDB	<code>http://localhost:8001</code>	Baza wektorowa (administracja)
Ollama	<code>http://localhost:11434</code>	Modele AI (diagnostyka)

Szybka ściągawka — codzienne uruchomienie

```
# 1. Uruchom Ollama (jeśli nie działa jako usługa w tle)
ollama serve

# 2. Przejdź do folderu projektu
cd C:\ścieżka\do\door-defects-chatbot

# 3. Uruchom aplikację
docker compose up

# 4. Otwórz przeglądarkę
# http://localhost:3000

# --- Po pracy ---

# 5. Zatrzymaj aplikację
docker compose down
```